

## RETO TÉCNICO 2: PATRONES DE DISEÑO ARQUITECTÓNICO

DOCUMENTO ÚNICO DE ENTREGABLES - Sistema: Hacienda

### IDENTIFICACIÓN DEL EQUIPO Y ROLES DE TRABAJO

| Integrante<br>(Nombre)   | ID        | Rol Asignado               | Frente de Responsabilidad                                                  |
|--------------------------|-----------|----------------------------|----------------------------------------------------------------------------|
| Diego Alejandro Villegas | 000553180 | Arquitecto Líder           | Detección de puntos rígidos, selección/descarte de patrones, diseño TO-BE. |
| Maria Fernanda Muñoz     | 000543839 | Arquitecta de Verificación | Demostrar que SOLID sigue en pie y preservación de comportamiento.         |
| Salomé Fonseca Arango    | 000248599 | Arquitecta de Riesgos      | Análisis de riesgo y plan de cambio/despliegue.                            |
| Maria del Pilar Mantilla | 000547259 | Arquitecta de Vistas       | Construcción de las dos vistas, bitácora IA y documento.                   |

### TABLA DE CONTENIDO

|                                                                               |    |
|-------------------------------------------------------------------------------|----|
| UNIVERSIDAD PONTIFICIA BOLIVARIANA - INGENIERÍA DE SISTEMAS                   | 1  |
| CURSO: ARQUITECTURA DE SOFTWARE                                               | 1  |
| RETO TÉCNICO 2: PATRONES DE DISEÑO ARQUITECTÓNICO                             | 1  |
| DOCUMENTO ÚNICO DE ENTREGABLES - Sistema: Hacienda                            | 1  |
| TABLA DE CONTENIDO                                                            | 1  |
| ACTIVIDAD 1 — ENCONTRAR LOS PUNTOS DE DOLOR DE LA ARQUITECTURA ACTUAL (AS-IS) | 2  |
| ACTIVIDAD 2 — DECIDIR SOBRE LOS PATRONES A INCORPORAR                         | 5  |
| ACTIVIDAD 3 — DISEÑAR EL TO-BE                                                | 11 |
| ACTIVIDAD 4 — DEMOSTRAR QUE SOLID SIGUE EN PIE                                | 12 |

|                                                      |    |
|------------------------------------------------------|----|
| ACTIVIDAD 5 — ANÁLISIS DE RIESGOS                    | 14 |
| ACTIVIDAD 6 — DOS VISTAS                             | 16 |
| 6.1 Vista para el Negocio                            | 16 |
| 6.2 Vista para el Equipo de Desarrollo (Guía Rápida) | 19 |

## ACTIVIDAD 1 — ENCONTRAR LOS PUNTOS DE DOLOR DE LA ARQUITECTURA ACTUAL (AS-IS)

*Instrucción del equipo: Mínimo 5 puntos de dolor. Al menos 3 encontrados sin IA. Medir el costo contando clases y archivos reales. Marcar al menos uno como "NO SE INTERVIENE".*

### Entregable 1.1: Análisis de puntos de dolor

| ID   | Nombre del Punto de Dolor                                                                 | Dónde (archivo / clase)                                                        | Qué lo hace rígido o caro (Síntoma)                                                                                                                                              | Cuánto cuesta hoy (N° Clases/Archivos)        | Prioridad                                                                                                                                                                                |
|------|-------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| P-01 | Creación rígida de subtipos de Res mediante switch y enum cerrado                         | ResController.cs<br><br>PersistenciaTxtService.cs                              | La creación concreta del tipo de res está concentrada en un switch. Agregar otro tipo obliga a modificar este controlador.                                                       | 4 a modificar + 1 a crear (5 archivos/clases) | Alta, es el núcleo del dominio ganadero. Cada cambio en la clasificación de animales propaga modificaciones forzadas en presentación, dominio e infraestructura.                         |
| P-02 | Bifurcación condicional (if/else), casteo explícito e interfaz rígida en jerarquía Vacuna | VacunaController.cs<br><br>PersistenciaTxtService.cs<br><br>ICreacionVacuna.cs | La clase base Vacuna.cs no encapsula el comportamiento ni los datos específicos de sus variantes. El controlador y el servicio de persistencia inspeccionan el tipo en tiempo de | 6 archivos y 6 clases/interfases/vistas       | Alta, La lógica de persistencia y de presentación está fuertemente acoplada a los tipos concretos de vacuna existentes, requiriendo reescribir bloques if/else, ternarios y casteos ante |

|             |                                                                               |                                                                  |                                                                                                                                                                                                                                                                                                                                                                                                   |                                                                     |                                                                                                                                                                                                          |
|-------------|-------------------------------------------------------------------------------|------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|             |                                                                               |                                                                  | ejecución mediante cadenas, pattern matching y casteo explícito ((Viva)vacuna).                                                                                                                                                                                                                                                                                                                   |                                                                     | cualquier nueva variante biológica.                                                                                                                                                                      |
| <b>P-03</b> | Interfaz monolítica de validación que obliga a lanzar NotImplementedException | IValidarInformacion.cs<br><br>Validacion.cs<br><br>Validaciones/ | La interfaz IValidarInformacion agrupa métodos de validación de cuatro conceptos de negocio totalmente independientes. Todas las clases validadoras se ven obligadas a heredar de la clase base Validacion y dejar métodos dummy con NotImplementedException. Agregar la validación de un nuevo elemento obliga a modificar la interfaz, la clase base y todas las clases validadoras existentes. | 6 archivos a modificar + 1 a crear (7 archivos y clases/interfases) | Media, es un síntoma evidente de acoplamiento innecesario que genera fragilidad y deuda técnica cada vez que una nueva entidad requiera validación formal.                                               |
| <b>P-04</b> | Coordinación e instanciación manual de eventos en servicios de aplicación     | ResService, PotreroService, VacunaService + publishers           | La generación y coordinación de notificaciones se encuentra asociada directamente a los servicios de aplicación y a publishers específicos. Cuando aparece una nueva reacción ante un acontecimiento, es necesario intervenir                                                                                                                                                                     | 3 archivos a modificar + 2 a crear (5 archivos y clases)            | Media-Alta, Cada nueva reacción o consumidor ante eventos del dominio (auditoría, alertas externas, telemetría) obliga a reabrir y modificar la lógica central de los métodos de servicio de aplicación, |

|             |                                                                |                                       |                                                                                                                                                                                                                                                                                                       |                    |                                                                                                                                                                                                                                                                                     |
|-------------|----------------------------------------------------------------|---------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|             |                                                                |                                       | <p>en la coordinación existente. Esto concentra en los servicios responsabilidades relacionadas con la notificación y aumenta la cantidad de código que debe revisarse ante nuevos consumidores.</p>                                                                                                  |                    | <p>rompiendo el principio de Abierto/Cerrado (OCP) y mezclando responsabilidades de coordinación de infraestructura con la lógica de negocio.</p>                                                                                                                                   |
| <b>P-05</b> | <p>Lógica de cálculo de venta condicionada por tipo de Res</p> | Venta.cs                              | <p>El cálculo del monto de una venta depende de múltiples condiciones que identifican el tipo concreto de Res. Cada nueva variante que requiera una fórmula de cálculo diferente obliga a modificar la lógica existente de venta, aumentando la cantidad de código que debe revisarse y probarse.</p> | 1 clase, 1 archivo | <p>Alta, porque el cálculo de venta es una decisión que puede crecer a medida que aparezcan nuevos tipos de Res o nuevas formas de comercialización. Mantener las condiciones en la lógica central obliga a modificar código existente cada vez que cambia la regla de cálculo.</p> |
| <b>P-06</b> | <p>Control de Flujo Basado en Excepciones (Anti-patrón)</p>    | Autenticacion.cs (AutorizarOperacion) | <p>Uso de excepciones como control de flujo: se lanza un "throw new Exception" cuando el acceso es EXITOSO. NO SE INTERVIENE (se marca como deuda técnica).</p>                                                                                                                                       | 1 clase, 1 archivo | <p>Baja — NO SE INTERVIENE en este reto. Aunque presenta un riesgo técnico, corregirlo implicaría modificar el mecanismo de autenticación y control de flujo existente,</p>                                                                                                         |

|  |  |  |  |  |                                                                                                                               |
|--|--|--|--|--|-------------------------------------------------------------------------------------------------------------------------------|
|  |  |  |  |  | aumentando el riesgo de alterar el comportamiento observable. Se registra como deuda técnica para una intervención posterior. |
|--|--|--|--|--|-------------------------------------------------------------------------------------------------------------------------------|

ACTIVIDAD 2 — DECIDIR SOBRE LOS PATRONES A INCORPORAR

Instrucción del equipo: Evaluar mínimo 6 patrones (2 de cada familia). Adoptar entre 3 y 5. Al menos 2 deben ser descartados con argumentos técnicos.

Entregable 2.1: Tabla de decisión de patrones

| Patrón evaluado | Familia    | Punto de dolor que atacaría    | Qué gana y qué cuesta                                                                                                                                                                                                 | Decisión   | Por qué                                                                                                                                                                                 |
|-----------------|------------|--------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Factory Method  | Creacional | P-01 (Switch en ResController) | <b>Gana:</b> centraliza la creación de subtipos de Res y evita que los consumidores conozcan directamente cada implementación.<br><b>Cuesta:</b> agrega una abstracción/fábricas y un nivel adicional de indirección. | ADOPTADO   | P-01 tiene exactamente el problema de creación de subtipos mediante decisiones concretas. Permite que agregar un nuevo tipo de Res no obligue a modificar el consumidor que lo utiliza. |
| Prototype       | Creacional | P-01 (Switch en ResController) | Gana: Elimina el switch clonando instancias preconfiguradas desde un registro. Cuesta:                                                                                                                                | DESCARTADO | Permite agregar nuevos tipos de animales simplemente registrando un nuevo prototipo al                                                                                                  |

|                         |                |                                                 |                                                                                                                                                                                                                                                                                                 |            |                                                                                                                                                                                                                                                                                 |
|-------------------------|----------------|-------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                         |                |                                                 | Obliga a implementar interfaces de clonación (ICloneable) en todo el modelo.                                                                                                                                                                                                                    |            | inicio, sin tocar el controlador (Cumple OCP).                                                                                                                                                                                                                                  |
| Chain of Responsibility | Comportamiento | <b>P-03 — Interfaz monolítica de validación</b> | <p><b>Gana:</b> permite separar las validaciones en objetos independientes y encadenarlas; agregar una nueva validación no obliga a modificar todas las validaciones existentes.</p> <p><b>Cuesta:</b> introduce una cadena de objetos, manejo del orden de validación y mayor indirección.</p> | ADOPTADO   | El problema actual concentra diferentes validaciones bajo un contrato rígido. Chain of Responsibility permite que cada validador se responsabilice de una condición específica y delegue al siguiente, facilitando agregar o retirar validaciones sin modificar las existentes. |
| Decorator               | Estructural    | <b>P-05 (Autenticación monolítica)</b>          | <p><b>Gana:</b> Añade validaciones envolviendo los servicios.</p> <p><b>Cuesta:</b> Genera cadenas de ejecución difíciles de depurar.</p>                                                                                                                                                       | DESCARTADO | El problema actual no es agregar responsabilidades dinámicas, sino que las reglas internas de acceso están quemadas. Strategy lo resuelve mejor.                                                                                                                                |
| Bridge                  | Estructural    | <b>P-02 (Casteos en jerarquía Vacuna)</b>       | <b>Gana:</b> Separa la abstracción de la vacuna de su implementación                                                                                                                                                                                                                            | DESCARTADO | Es el patrón perfecto para evitar inspeccionar tipos en tiempo de                                                                                                                                                                                                               |

|          |                |                                                                                             |                                                                                                                                                                                                                                   |          |                                                                                                                                                                                                                                                                                        |
|----------|----------------|---------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|          |                |                                                                                             | <p>biológica, eliminando casteos ((Viva)vacuna).</p> <p><b>Cuesta:</b><br/>Aumenta la complejidad inicial de la jerarquía.</p>                                                                                                    |          | <p>ejecución. El puente permite que ambas dimensiones (concepto vacuna y biología) varíen independientemente.</p>                                                                                                                                                                      |
| Strategy | Comportamiento | <b>P-05 (Múltiples condicionales para calcular el precio de Venta según el tipo de Res)</b> | <p><b>Gana:</b><br/>Encapsula las reglas de cálculo matemático en clases separadas (OCP).</p> <p><b>Cuesta:</b><br/>Aumenta la cantidad de clases e interfaces pequeñas en el dominio.</p>                                        | ADOPTADO | <p>El cálculo actual depende de condiciones asociadas al tipo de Res. Strategy permite trasladar cada fórmula a una implementación independiente de <code>IEstrategiaCalculoVenta</code>, reduciendo las modificaciones sobre la lógica central de Venta.</p>                          |
| Observer | Comportamiento | <b>P-04 (Eventos en servicios)</b>                                                          | <p><b>Gana:</b> permite que múltiples consumidores reaccionen ante un acontecimiento sin que el servicio que lo genera tenga que coordinar directamente cada reacción.</p> <p><b>Cuesta:</b> introduce indirección y requiere</p> | ADOPTADO | <p>P-04 concentra la coordinación de las notificaciones en los servicios actuales. Observer permite separar la generación del acontecimiento de las reacciones que otros componentes deben ejecutar, de manera que un nuevo consumidor pueda incorporarse mediante suscripción sin</p> |

|                 |                |                                  |                                                                                                                                                                                                                                            |            |                                                                                                                                                                                                                                                                 |
|-----------------|----------------|----------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                 |                |                                  | administrar las suscripciones y los datos que se entregan a los consumidores.                                                                                                                                                              |            | modificar la lógica principal que produce el evento.                                                                                                                                                                                                            |
| Template Method | Comportamiento | <b>P-03(Interfaz monolítica)</b> | <b>Gana:</b> permite definir una estructura común de validación y variar pasos concretos.<br><b>Cuesta:</b> aumenta el acoplamiento mediante herencia y puede generar problemas de LSP si las subclases no pueden cumplir todos los pasos. | DESCARTADO | P-03 no necesita necesariamente una secuencia común de pasos; el problema principal es que una interfaz obliga a implementaciones a declarar operaciones que no necesitan. Template Method podría trasladar el problema a una jerarquía de herencia más rígida. |

## Entregable 2.2: Bitácora de decisiones frente a la IA

(Mínimo 10 registros detallando la interacción)

| ID   | Qué consultaron                                                       | Qué propuso la herramienta          | Qué hicieron (Aceptar/Corregir/Rechazar) | Argumento propio y evidencia                                                                                                                                          |
|------|-----------------------------------------------------------------------|-------------------------------------|------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| B-01 | Patrones para eliminar el switch de creación en ResController (P-01). | Propuso Prototype y Factory Method. | Aceptar Factory Method                   | Elegimos Factory Method porque centraliza la creación lógica de las reses según sus parámetros de negocio, en lugar de clonar objetos en memoria como hace Prototype. |



|      |                                                                                                                         |                                                                 |                                    |                                                                                                                                                                                                                                                                                                                                                         |
|------|-------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------|------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| B-02 | Manejo de parámetros variables al crear Reses con Factory Method.                                                       | Sugirió usar múltiples firmas en la interfaz de la fábrica.     | Corregir (Uso de DTO)              | Usar múltiples firmas rompe el principio LSP. Inyectaremos un objeto unificado <b>ResDTO</b> que contenga todos los parámetros posibles para estandarizar la creación.                                                                                                                                                                                  |
| B-03 | Separación de la interfaz monolítica IValidarInformacion (P-03).                                                        | Sugirió Chain of Responsibility o Template Method.              | Aceptar Chain of Responsibility    | Chain of Responsibility nos permite tener validadores pequeños y aislados. Si la validación falla, se corta la cadena, lo cual es ideal para reglas de negocio secuenciales.                                                                                                                                                                            |
| B-04 | Cómo orquestar la cadena de responsabilidad sin acoplar el dominio.                                                     | Propuso instanciar la cadena directamente en los controladores. | Corregir (Inyección en Program.cs) | Ensamblaremos el orden de los eslabones de la cadena directamente en el contenedor de dependencias ( <b>Program.cs</b> ) para proteger el principio DIP                                                                                                                                                                                                 |
| B-05 | Patrones para resolver la lógica condicional utilizada para calcular el monto de una Venta según el tipo de Res (P-05). | Sugirió Decorator y Strategy.                                   | Aceptar Strategy                   | Elegimos Strategy porque el problema identificado en P-05 corresponde a una variación de comportamiento: el cálculo del monto puede cambiar según el tipo de Res. Encapsular cada fórmula en una implementación de IEstrategiaCalculoVenta permite agregar o modificar una estrategia sin introducir nuevos if/switch en la lógica central de Venta. La |

|      |                                                                                                                                                     |                                                                                  |                                   |                                                                                                                                                                                                                                                                                                                                                                                             |
|------|-----------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------|-----------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|      |                                                                                                                                                     |                                                                                  |                                   | evidencia del AS-IS es la presencia de condiciones que determinan el cálculo según el tipo concreto de Res.                                                                                                                                                                                                                                                                                 |
| B-06 | Cómo seleccionar la estrategia de cálculo correspondiente al tipo de Res sin volver a introducir condicionales en la lógica central de Venta.       | Sugirió inyectar todas las estrategias y seleccionarlas usando un bloque switch. | Corregir (Factory de Estrategias) | Consideramos que utilizar nuevamente un switch trasladaría el mismo problema identificado en P-05 a otro punto del sistema. Por ello, la selección de la estrategia debe realizarse mediante una configuración que asocie el tipo de Res con su implementación de IEstrategiaCalculoVenta, manteniendo la clase Venta dependiente de la abstracción y no de las implementaciones concretas. |
| B-07 | Implementación de eventos desacoplados para eliminar la coordinación manual de notificaciones en ResService, PotreroService y VacunaService (P-04). | Sugirió utilizar Observer mediante IObservable<T> o Mediator.                    | Aceptar Observer (Simplificado)   | Elegimos Observer porque P-04 requiere que un mismo acontecimiento pueda generar reacciones independientes sin que el servicio que lo produce tenga que coordinar directamente cada consumidor. No utilizaremos una infraestructura reactiva adicional; utilizaremos el mecanismo de publicación y suscripción representado en el TO-BE junto con los                                       |

|      |                                                                                                                      |                                                                                                                        |                                    |                                                                                                                                                                                                                                                                                                                                                                                                                                    |
|------|----------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------|------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|      |                                                                                                                      |                                                                                                                        |                                    | objetos de datos de los eventos. Esto permite agregar consumidores sin modificar la lógica principal que genera el acontecimiento.                                                                                                                                                                                                                                                                                                 |
| B-08 | Cómo evitar que los consumidores de los eventos dependan directamente de las entidades completas del sistema (P-04). | Sugirió enviar un payload genérico para evitar que los observadores recibieran directamente las entidades del dominio. | Corregir (Uso de Payload genérico) | La propuesta se ajustó al diseño TO-BE. En lugar de entregar directamente las entidades completas a los consumidores, los acontecimientos transportan la información necesaria mediante los objetos de datos definidos para cada evento. De esta manera, los consumidores reciben únicamente la información necesaria para reaccionar ante el acontecimiento y se reduce su dependencia de la estructura interna de las entidades. |
| B-09 | Control de excepciones para flujo exitoso en la autenticación (P-06).                                                | Propuso el patrón Result (monad) o reestructurar retornos booleanos.                                                   | Rechazar (Deuda Técnica)           | Modificar el control de flujo excede el alcance del reto. Como el equipo acordó, no se interviene y se marca como deuda técnica para el futuro.                                                                                                                                                                                                                                                                                    |
| B-10 | Manejo de tipos biológicos en la jerarquía de Vacuna (P-02).                                                         | Propuso Bridge para separar abstracción de implementación.                                                             | Rechazar Bridge                    | Aunque es útil, decidimos enfocar el esfuerzo estructural en la validación y autenticación (P-03 y P-05) priorizando Chain y Strategy por el impacto                                                                                                                                                                                                                                                                               |

|  |  |  |  |                |
|--|--|--|--|----------------|
|  |  |  |  | en el negocio. |
|--|--|--|--|----------------|

ACTIVIDAD 3 — DISEÑAR EL TO-BE

Entregable 3.1: Tabla de cambio estructural

| ID   | Elemento            | Estado        | Qué hacía antes                                      | Qué hace ahora                                                                           | Quién dependía de él y cómo se reconecta                                                            |
|------|---------------------|---------------|------------------------------------------------------|------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------|
| E-01 | ResController       | Se transforma | Decidía mediante switch qué subtipo de Res crear     | Solicita la creación mediante la fábrica correspondiente sin conocer el subtipo concreto | El controlador deja de depender directamente de Ternero, Cebon y Novillo; se conecta con ResFactory |
| E-02 | ResFactory          | Entra         | N/A                                                  | Define la operación de creación de una Res                                               | Es utilizada por el punto de creación de reses                                                      |
| E-03 | TerneroFactory      | Entra         | N/A                                                  | Crea Ternero                                                                             | Implementa ResFactory                                                                               |
| E-04 | CebonFactory        | Entra         | N/A                                                  | Crea Cebon                                                                               | Implementa ResFactory                                                                               |
| E-05 | NovilloFactory      | Entra         | N/A                                                  | Crea Novillo                                                                             | Implementa ResFactory                                                                               |
| E-06 | IValidarInformacion | Sale          | Agrupaba todas las operaciones de validación         | Se reemplaza por validadores especializados                                              | Las reglas que utilizaban la interfaz pasan a utilizar la cadena                                    |
| E-07 | Validacion          | Sale          | Clase base que obligaba a implementar operaciones no | Se elimina como mecanismo común de validación                                            | Las responsabilidades pasan a handlers específicos                                                  |

|      |                                                                            |       |                                                                 |                                                                                          |                                                               |
|------|----------------------------------------------------------------------------|-------|-----------------------------------------------------------------|------------------------------------------------------------------------------------------|---------------------------------------------------------------|
|      |                                                                            |       | necesarias                                                      |                                                                                          |                                                               |
| E-08 | ValidarRes,<br>ValidarPotrero,<br>ValidarVacuna                            | Sale  | Validaba dentro de la jerarquía anterior                        | Su responsabilidad pasa a ValidarVacunaHandler, ValidarResHandler, ValidarPotreroHandler | Se conecta como eslabón de la cadena                          |
| E-09 | ValidadorHandler                                                           | Entra | N/A                                                             | Define el contrato y referencia al siguiente validador                                   | Es utilizado por los handlers concretos                       |
| E-10 | IEstrategiaCalculoVenta                                                    | Entra | N/A                                                             | Define el contrato para calcular el monto de venta                                       | Venta utiliza esta abstracción                                |
| E-11 | EstrategiaVentaTernero,<br>EstrategiaVentaCebon,<br>EstrategiaVentaNovillo | Entra | El cálculo estaba condicionado                                  | Calcula el precio correspondiente a Novillo, Ternero y Res                               | Implementa IEstrategiaCalculoVenta                            |
| E-12 | PublisherPesoMin                                                           | Sale  | Coordinaba/notificaba manualmente eventos relacionados con peso | Se reemplaza por el nuevo mecanismo de eventos                                           | Los consumidores pasan a recibir información mediante eventos |
| E-13 | PublisherPotreroMitad,<br>PublisherPotreroLleno                            | Sale  | Notificaba manualmente el estado del potrero                    | Se reemplaza por eventos                                                                 | Los consumidores se suscriben al evento correspondiente       |

### Entregable 3.3: Fichas por cada patrón adoptado

*(Copiar y pegar esta ficha por cada patrón ADOPTADO, máximo 1 página por ficha)*

#### Ficha: Factory Method

- **Patrón y punto de dolor:** Factory Method — Resuelve P-01, relacionado con la creación rígida de subtipos de Res mediante switch y decisiones concretas en ResController.
- **Alternativas evaluadas:**
  - No hacer nada y mantener el switch, lo que obligaría a modificar el punto de creación cada vez que aparezca un nuevo subtipo.
  - Prototype, descartado porque introduce clonación de objetos y una infraestructura de prototipos que no corresponde con la necesidad principal del sistema, que es encapsular la creación según el tipo de animal.
- **Qué sale y qué entra:**

**Salen:** decisiones switch/condicionales concentradas en el punto de creación.

**Entran:** ResFactory, TerneroFactory, CebonFactory y NovilloFactory.
- **Cómo se relaciona:** ResFactory define CrearRes() y las fábricas concretas implementan la creación del subtipo correspondiente. El consumidor solicita la creación sin depender directamente de la implementación concreta.
- **Impacto:** Se crean 4 clases relacionadas con la fábrica. Se reduce la necesidad de modificar el consumidor cuando aparezca un nuevo subtipo de Res.
- **Qué cuesta:** Aumenta el número de clases y agrega un nivel adicional de indirección entre quien solicita la creación y el objeto creado.
- **Origen:** B-01.

#### Ficha: Strategy

- **Patrón y punto de dolor:** Strategy - Resuelve el P-05 (Cálculo rígido de montos en Ventas).
- **Alternativas evaluadas:**
  - No hacer nada y usar un bloque switch en la clase Venta, lo cual violaría OCP cada vez que se agregue un nuevo tipo de animal o producto (SC-1).
  - Template Method, descartado porque la variación no es de pasos de un algoritmo, sino de una fórmula de cálculo directa.
- **Qué sale y qué entra:**
  - Salen: Múltiples if/switch para calcular precios.
  - Entran: Interfaz IEstrategiaCalculoVenta y sus implementaciones concretas (EstrategiaVentaTernero, Novillo, Cebon).
- **Cómo se relaciona:** La clase Venta se compone (Agregación) de IEstrategiaCalculoVenta recibéndola a través de su método ProcesarVenta().
- **Impacto:** Clases creadas: 4. Prepara el sistema para soportar ventas de derivados (SC-1)

sin tocar el core de facturación.

- **Qué cuesta:** Mayor indirección a la hora de rastrear cómo se calculó un precio específico en tiempo de ejecución.
- **Origen:** B-05.

#### Ficha: Chain Of Responsibility

- **Patrón y punto de dolor:** Chain of Responsibility — Resuelve P-03, correspondiente a la interfaz monolítica IValidarInformacion.
- **Alternativas evaluadas:**
  - No hacer nada y mantener la interfaz actual, conservando métodos que algunas implementaciones no necesitan.
  - Template Method, descartado porque el problema no consiste en compartir una secuencia fija de pasos, sino en separar validaciones independientes.
- **Qué sale y qué entra:**
  - Salen: IValidarInformacion, Validacion, ValidarRes, ValidarPotrero y ValidarVacuna como estructura común.
  - Entran: ValidadorHandler, ValidarResHandler, ValidarPotreroHandler y ValidarVacunaHandler.
- **Cómo se relaciona:** Los handlers se encadenan mediante \_siguiente. Cada handler realiza su validación específica y delega al siguiente cuando corresponde. La cadena se ensambla fuera de las clases de dominio.
- **Impacto:** Se crean 4 componentes principales para la cadena y se eliminan las responsabilidades innecesarias de la estructura anterior. Una nueva validación puede agregarse como un nuevo handler.
- **Qué cuesta:** Se introduce una secuencia de objetos y mayor indirección. El orden de los validadores debe configurarse correctamente.
- **Origen:** B-03 y B-04.

#### Ficha: Observer

- **Patrón y punto de dolor:** Observer — Resuelve P-04, relacionado con la coordinación manual de eventos en los servicios de aplicación.
- **Alternativas evaluadas:**
  - No hacer nada y continuar coordinando directamente las reacciones desde los servicios.
  - Mediator, descartado porque introduciría un componente central adicional para coordinar las comunicaciones, mientras que el problema identificado requiere que varios consumidores puedan reaccionar independientemente ante un acontecimiento.
- **Qué sale y qué entra:**
  - Salen: los publishers específicos utilizados para coordinar manualmente las

notificaciones.

- Entran: EventArgs, ResEventArgs, PotreroEventArgs y VacunaEventArgs, junto con el mecanismo de publicación/suscripción.
- **Cómo se relaciona:** Los servicios generan acontecimientos y los consumidores interesados se suscriben a ellos. La información del acontecimiento se transporta mediante objetos EventArgs específicos, evitando que el consumidor dependa de la implementación completa del publicador.
- **Impacto:** Se incorporan objetos para representar los datos de los eventos y se modifica la forma en que los servicios notifican acontecimientos. Nuevos consumidores pueden suscribirse sin modificar la lógica principal que produce el evento.
- **Qué cuesta:** Aumenta la indirección y se debe administrar correctamente la suscripción de los consumidores. Un fallo en un consumidor puede generar comportamientos secundarios que deben ser controlados.
- **Origen:** B-07 y B-08.

## ACTIVIDAD 4 — DEMOSTRAR QUE SOLID SIGUE EN PIE

### Entregable 4.1: Matriz de verificación

| Patrón adoptado         | SRP      | OCP      | LSP                        | ISP      | DIP                        |
|-------------------------|----------|----------|----------------------------|----------|----------------------------|
| Factory Method          | REFUERZA | REFUERZA | TENSIONADO PERO COMPENSADO | REFUERZA | REFUERZA                   |
| Chain of Responsibility | REFUERZA | REFUERZA | REFUERZA                   | REFUERZA | TENSIONADO PERO COMPENSADO |
| Strategy                | REFUERZA | REFUERZA | REFUERZA                   | REFUERZA | TENSIONADO PERO COMPENSADO |
| Observer                | REFUERZA | REFUERZA | REFUERZA                   | REFUERZA | TENSIONADO PERO COMPENSADO |

Evidencia de impactos en principios (Justificación de valores distintos a Neutro):



- **Factory Method - LSP (Tensionado pero compensado):** Si un nuevo tipo de Res en el futuro requiere parámetros de inicialización radicalmente distintos que los demás, la interfaz estándar de la fábrica no encajará, forzando a la fábrica a ignorar parámetros y rompiendo la sustitución ideal. Se compensa inyectando un objeto de contexto unificado (ej. ResDTO) que encapsula todos los datos, estandarizando la firma de creación
- **Chain of Responsibility - DIP (Tensionado pero compensado):** La interfaz monolítica desaparece, pero en algún lugar del código se debe instanciar y establecer el orden de los validadores (la cadena), lo que podría acoplar a un cliente de alto nivel con las clases validadoras concretas. Se compensa realizando el ensamblaje de la cadena enteramente en el archivo Program.cs usando el contenedor de Inyección de Dependencias, aislando al dominio de las clases concretas.
- **Strategy - DIP (Tensionado pero compensado):** Las estrategias de cálculo de venta deben depender únicamente de las abstracciones necesarias para realizar su responsabilidad y no de implementaciones concretas externas. Existe una tensión porque la selección y configuración de la estrategia requiere asociar el tipo de Res con una implementación concreta. Se compensa haciendo que Venta dependa exclusivamente de IEstrategiaCalculoVenta y que la configuración de las implementaciones concretas se realice fuera de la lógica central de cálculo.
- **Observer - DIP (Tensionado pero compensado):** Si un Observador (ej. AlertaExternaObserver) necesita demasiada información para hacer su trabajo, podría verse tentado a hacer un casteo explícito ("downcasting") de la clase publicadora para extraer los datos, acoplándose fuertemente a su implementación. Se compensa enviando toda la información requerida de manera agnóstica a través de un objeto de transferencia genérico (EventoPayload).

## Entregable 4.2: Evidencias de que el comportamiento no cambia

Para el Reto 2, la suite de pruebas contenida en PruebasCaracterizacion.cs fue evolucionada en comparación con la versión del Reto 1. La razón técnica de este cambio se justifica bajo dos principios de ingeniería:

1. **Evolución de Impresión a Auditoría Automática de Salidas (Motor de Aserción):**
  - *En el Reto 1:* Las pruebas se limitaban a imprimir el resultado en la consola mediante `Console.WriteLine()`, requiriendo que un ser humano inspeccionara visualmente si el texto producido era correcto.

Salida del entregable pasado:

```
terminal dotnet +
... el propiedad como un valor que acepta valores NULL.
/Volumes/Diego/Universidad/Semestre_6/Arquitectura_de_software/HaciendaGanadera/proyecto_hacienda/p_mvchacienda/Models/LoginViewModel.cs(12,23): warning CS8604: El argumento de referencia nulo para el parámetro "password" que no acepta valores NULL debe contener un valor distinto de NULL al salir del constructor. Considere la posibilidad de agregar el modificador 'NotNull' al parámetro "password" para indicar que no se permite un valor NULL.
/Volumes/Diego/Universidad/Semestre_6/Arquitectura_de_software/HaciendaGanadera/proyecto_hacienda/p_mvchacienda/Controllers/VentaController.cs(41,36): warning CS8604: El argumento de referencia nulo para el parámetro "potreroId" en "string IVentaService.VenderRes(string potreroId, string nombreRes, uint monto)".
/Volumes/Diego/Universidad/Semestre_6/Arquitectura_de_software/HaciendaGanadera/proyecto_hacienda/p_mvchacienda/Controllers/VentaController.cs(42,36): warning CS8604: El argumento de referencia nulo para el parámetro "nombreRes" en "string IVentaService.VenderRes(string potreroId, string nombreRes, uint monto)".
/Volumes/Diego/Universidad/Semestre_6/Arquitectura_de_software/HaciendaGanadera/proyecto_hacienda/p_mvchacienda/Controllers/VentaController.cs(43,41): warning CS8604: El argumento de referencia nulo para el parámetro "s" en "uint uint.Parse(string s)".
/Volumes/Diego/Universidad/Semestre_6/Arquitectura_de_software/HaciendaGanadera/proyecto_hacienda/p_mvchacienda/Controllers/VentaController.cs(45,41): warning CS8604: El argumento de referencia nulo para el parámetro "potreroId" en "string IVentaService.VenderRes(string potreroId, string nombreRes, uint monto)".
/Volumes/Diego/Universidad/Semestre_6/Arquitectura_de_software/HaciendaGanadera/proyecto_hacienda/p_mvchacienda/Controllers/VentaController.cs(45,52): warning CS8604: El argumento de referencia nulo para el parámetro "nombreRes" en "string IVentaService.VenderRes(string potreroId, string nombreRes, uint monto)".
/Volumes/Diego/Universidad/Semestre_6/Arquitectura_de_software/HaciendaGanadera/proyecto_hacienda/p_mvchacienda/Servicios/ResService.cs(72,20): warning CS8604: El argumento de referencia nulo para el parámetro "res" en "uint uint.Parse(string res)".
valor devuelto de referencia nulo.
=== CASOS DE CARACTERIZACIÓN - SISTEMA REFACTORIZADO (TO-BE) ===

CC-01 -> RESULTADO: La res 'Vaca1' fue añadida al potrero 'P1'.
CC-02 -> EXCEPCIÓN: La res 'Vaca2' no cumple las condiciones de peso/edad para su categoría.
CC-03 -> RESULTADO: La res 'Vaca3' fue añadida al potrero 'P2'.
CC-04 -> EXCEPCIÓN: La res 'Vaca4' no cumple las condiciones de peso/edad para su categoría.
CC-05 -> RESULTADO: La res 'Vaca5' fue añadida al potrero 'P3'.
CC-06 -> EXCEPCIÓN: El potrero 'P4' alcanzó su capacidad máxima (150 reses).
CC-07 -> EXCEPCIÓN: El nombre de la res no puede estar vacío.
CC-08 -> RESULTADO: Res encontrada: Vaca1, Edad: 10, Peso: 200

Info: Microsoft.Hosting.Lifetime[14]
      Now listening on: http://localhost:5231
Info: Microsoft.Hosting.Lifetime[0]
      Application started. Press Ctrl+C to shut down.
Info: Microsoft.Hosting.Lifetime[0]
      Hosting environment: Development
Info: Microsoft.Hosting.Lifetime[0]
      Content root path: /Volumes/Diego/Universidad/Semestre_6/Arquitectura_de_software/HaciendaGanadera/proyecto_hacienda/p_mvchacienda
```

- *En el Reto 2:* Se implementó un motor de aserción automático (ValidarYComparar()). Este motor almacena la cadena de texto exacta de la **Línea Base (AS-IS)** y la compara mediante coincidencia estricta (.Equals()) contra el valor **Obtenido (TO-BE)**. Si el comportamiento refactorizado genera una sola variación en los mensajes o excepciones, la consola arroja inmediatamente una alerta visual en rojo de [X FALLÓ - REGRESIÓN DETECTADA]. Si la coincidencia es absoluta, se certifica con [✓ PASÓ - AS-IS CONSERVADO].

Evidencia de ese orquestador:

```
projecto_hacienda
...
namespace p_mvchacienda
{
    public static class PruebasCaracterizacion
    {
        // --- ORQUESTADOR CON CHAIN OF RESPONSIBILITY ---
        static string AgregarConValidacion(Potrero potrero, Res res)
        {
            if (res == null || string.IsNullOrWhiteSpace(res.Nombre)) {
                throw new ArgumentException("El nombre de la res no puede estar vacío.");
            }
            if (potrero.buscar_res(res.Nombre) != null) {
                throw new InvalidOperationException($"Ya existe una res con el nombre '{res.Nombre}' en el potrero '{potrero.Nombre}'");
            }
            // Invocación del Patrón Handler
            ValidadorHandler cadenaValidacion = new ValidarPotreroHandler();
            cadenaValidacion.SetNext(new ValidarResHandler());

            cadenaValidacion.Validar(potrero);
            cadenaValidacion.Validar(res);

            return potrero.anadir_res(res);
        }
    }
}
```

```
10 namespace p_mvchacienda
12 public static class PruebasCaracterizacion
140
141 // --- MOTOR DE ASERSION Y AUDITORÍA DE SALIDAS ---
142
143 static void ValidarYComparar(string id, string descripcion, string esperado, Func<string> accion)
144 {
145     string obtenido;
146     try {
147         obtenido = accion();
148     }
149     catch (Exception ex) {
150         obtenido = $"EXCEPCION: {ex.Message}";
151     }
152
153     // Normalización básica para ignorar diferencias diminutas de espacios
154     bool coinciden = obtenido.Trim().Equals(esperado.Trim(), StringComparison.OrdinalIgnoreCase);
155
156     if (coinciden)
157     {
158         Console.ForegroundColor = ConsoleColor.Green;
159         Console.WriteLine($"[✓ PASÓ - AS-IS CONSERVADO] {id}: {descripcion}");
160         Console.ResetColor();
161         Console.WriteLine($"--> Resultado: {obtenido}\n");
162     }
163     else
164     {
165         Console.ForegroundColor = ConsoleColor.Red;
166         Console.WriteLine($"[X FALLÓ - REGRESIÓN DETECTADA] {id}: {descripcion}");
167         Console.ResetColor();
168         Console.WriteLine($"--> Esperado: {esperado}");
169         Console.WriteLine($"--> Obtenido: {obtenido}\n");
170     }
171 }
```

Salida nueva respetando comportamiento:

```
PS C:\Users\mafem\OneDrive\Escritorio\Mafe\UPB\Semestre 6\Arquitectura de Software\Evaluaciones\Evaluación 2\Hacienda\proyecto_hacienda\p_mvchacienda> dotnet run
Usando la configuración de inicio de C:\Users\mafem\OneDrive\Escritorio\Mafe\UPB\Semestre 6\Arquitectura de Software\Evaluaciones\Evaluación 2\Hacienda\proyecto_hacienda\p_mvchacienda\Properties\launchSettings.json...
Compilando...

=====
EVIDENCIA DE REGRESIÓN: AUDITORÍA DE COMPORTAMIENTO (AS-IS vs TO-BE)
=====

[✓ PASÓ - AS-IS CONSERVADO] CC-01: Añadir Ternero Vaca1 válido a Potrero P1
--> Resultado: La res 'Vaca1' fue añadida al potrero 'P1'.

[✓ PASÓ - AS-IS CONSERVADO] CC-02: Rechazar Ternero Vaca2 con edad inválida para categoría
--> Resultado: EXCEPCION: La res 'Vaca2' no cumple las condiciones de peso/edad para su categoría.

[✓ PASÓ - AS-IS CONSERVADO] CC-03: Añadir Cebón Vaca3 válido a Potrero P2
--> Resultado: La res 'Vaca3' fue añadida al potrero 'P2'.

[✓ PASÓ - AS-IS CONSERVADO] CC-04: Rechazar Cebón Vaca4 por edad fuera de rango
--> Resultado: EXCEPCION: La res 'Vaca4' no cumple las condiciones de peso/edad para su categoría.

[✓ PASÓ - AS-IS CONSERVADO] CC-05: Añadir Novillo Vaca5 válido a Potrero P3
--> Resultado: La res 'Vaca5' fue añadida al potrero 'P3'.

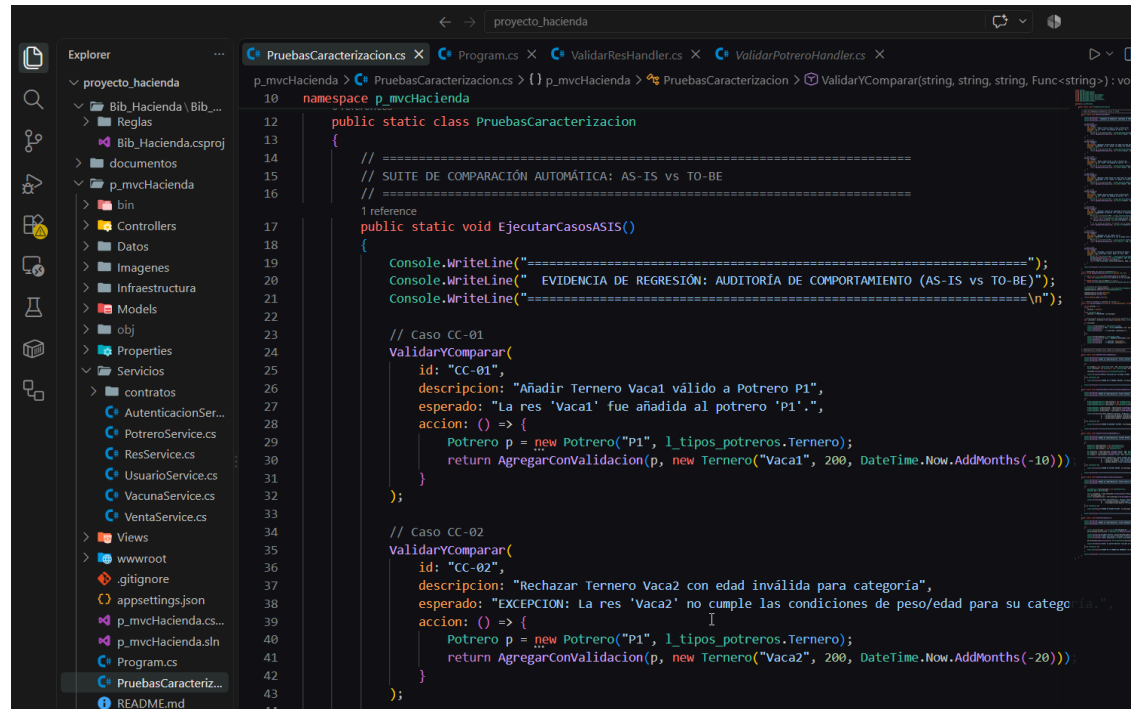
[✓ PASÓ - AS-IS CONSERVADO] CC-06: Rechazar ingreso por capacidad máxima alcanzada en Potrero P4
--> Resultado: EXCEPCION: El potrero 'P4' alcanzó su capacidad máxima (150 reses).

[✓ PASÓ - AS-IS CONSERVADO] CC-07: Rechazar res con nombre vacío
--> Resultado: EXCEPCION: El nombre de la res no puede estar vacío.

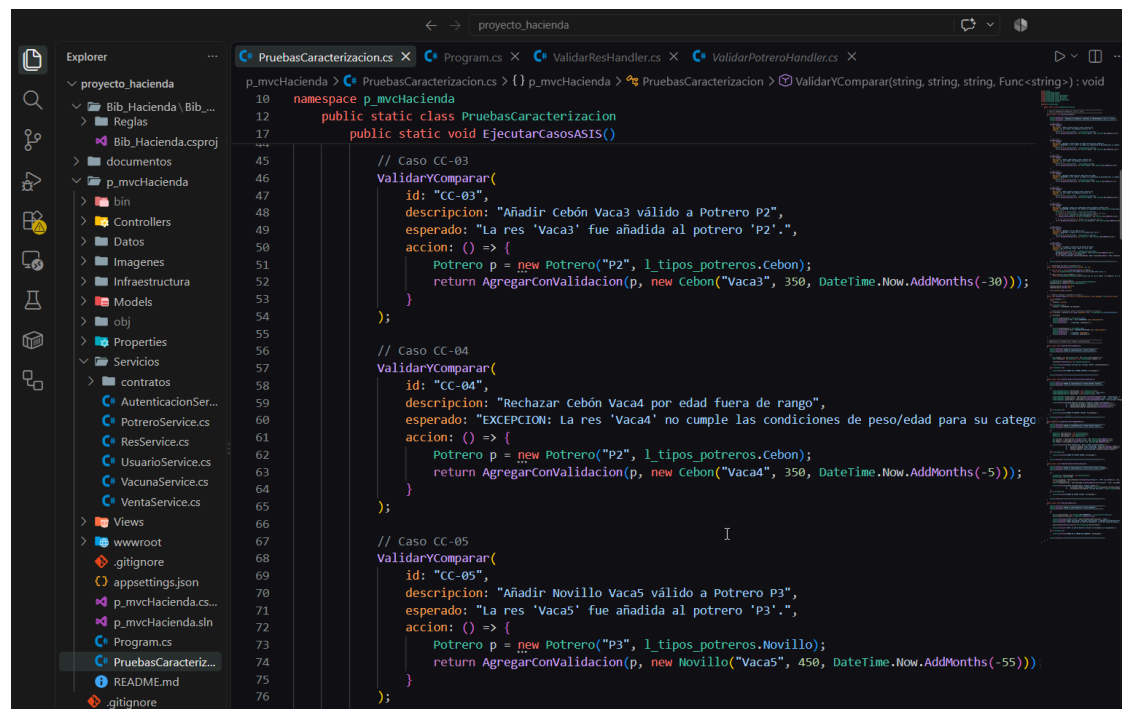
[✓ PASÓ - AS-IS CONSERVADO] CC-08: Búsqueda y consulta de atributos de Vaca1
--> Resultado: Res encontrada: Vaca1, Edad: 10, Peso: 200

=====
```

- Evidencia de cómo se encuentra el nuevo PruebasCaracterizacion.cs:



```
10 namespace p_mvchacienda
11
12 public static class PruebasCaracterizacion
13 {
14     // =====
15     // SUITE DE COMPARACIÓN AUTOMÁTICA: AS-IS vs TO-BE
16     // =====
17     // 1 reference
18     public static void EjecutarCasosASIS()
19     {
20         Console.WriteLine("=====");
21         Console.WriteLine(" EVIDENCIA DE REGRESIÓN: AUDITORÍA DE COMPORTAMIENTO (AS-IS vs TO-BE)");
22         Console.WriteLine("=====");
23
24         // Caso CC-01
25         ValidarYComparar(
26             id: "CC-01",
27             descripcion: "Añadir Ternero Vacal válido a Potrero P1",
28             esperado: "La res 'Vacal' fue añadida al potrero 'P1'.",
29             accion: () => {
30                 Potrero p = new Potrero("P1", 1_tipos_potreros.Ternero);
31                 return AgregarConValidacion(p, new Ternero("Vacal", 200, DateTime.Now.AddMonths(-10)));
32             }
33         );
34
35         // Caso CC-02
36         ValidarYComparar(
37             id: "CC-02",
38             descripcion: "Rechazar Ternero Vacal con edad inválida para categoría",
39             esperado: "EXCEPCION: La res 'Vacal' no cumple las condiciones de peso/edad para su categoría.",
40             accion: () => {
41                 Potrero p = new Potrero("P1", 1_tipos_potreros.Ternero);
42                 return AgregarConValidacion(p, new Ternero("Vacal", 200, DateTime.Now.AddMonths(-20)));
43             }
44         );
45     }
46 }
```



```
45
46 // Caso CC-03
47 ValidarYComparar(
48     id: "CC-03",
49     descripcion: "Añadir Cebón Vacal3 válido a Potrero P2",
50     esperado: "La res 'Vacal3' fue añadida al potrero 'P2'.",
51     accion: () => {
52         Potrero p = new Potrero("P2", 1_tipos_potreros.Cebon);
53         return AgregarConValidacion(p, new Cebon("Vacal3", 350, DateTime.Now.AddMonths(-30)));
54     }
55 );
56
57 // Caso CC-04
58 ValidarYComparar(
59     id: "CC-04",
60     descripcion: "Rechazar Cebón Vacal4 por edad fuera de rango",
61     esperado: "EXCEPCION: La res 'Vacal4' no cumple las condiciones de peso/edad para su categoría.",
62     accion: () => {
63         Potrero p = new Potrero("P2", 1_tipos_potreros.Cebon);
64         return AgregarConValidacion(p, new Cebon("Vacal4", 350, DateTime.Now.AddMonths(-5)));
65     }
66 );
67
68 // Caso CC-05
69 ValidarYComparar(
70     id: "CC-05",
71     descripcion: "Añadir Novillo Vacal5 válido a Potrero P3",
72     esperado: "La res 'Vacal5' fue añadida al potrero 'P3'.",
73     accion: () => {
74         Potrero p = new Potrero("P3", 1_tipos_potreros.Novillo);
75         return AgregarConValidacion(p, new Novillo("Vacal5", 450, DateTime.Now.AddMonths(-55)));
76     }
77 );
78 }
```

```

p_mvcHacienda > PruebasCaracterizacion.cs > {} p_mvcHacienda > PruebasCaracterizacion > ValidarYComparar(string, string, string, Func<string>): void
10 namespace p_mvcHacienda
12 public static class PruebasCaracterizacion
17 public static void EjecutarCasosASIS()
76 );
77
78 // Caso CC-06
79 ValidarYComparar(
80 id: "CC-06",
81 descripcion: "Rechazar ingreso por capacidad máxima alcanzada en Potrero P4",
82 esperado: $"EXCEPCION: El potrero 'P4' alcanzó su capacidad máxima ({ReglaPotrero.max_rese
83 accion: () => {
84     Potrero p = new Potrero("P4", l_tipos_potreros.Novillo);
85     // Llenamos el potrero hasta su límite estricto de negocio (50 reses)
86     for (int i = 0; i < ReglaPotrero.max_reses_potrero; i++) {
87         AgregarConValidacion(p, new Novillo($"Res{i}", 450, DateTime.Now.AddMonths(-55)));
88     }
89     // La res número 51 detonará el Handler con la excepción exacta
90     return AgregarConValidacion(p, new Novillo("VacaExtra", 450, DateTime.Now.AddMonths(-5
91 });
92
93 // Caso CC-07
94 ValidarYComparar(
95 id: "CC-07",
96 descripcion: "Rechazar res con nombre vacío",
97 esperado: "EXCEPCION: El nombre de la res no puede estar vacío.",
98 accion: () => {
99     Potrero p = new Potrero("P1", l_tipos_potreros.Ternero);
100     return AgregarConValidacion(p, new Ternero("", 200, DateTime.Now.AddMonths(-10)));
101 }
102
103 );

```

```

p_mvcHacienda > PruebasCaracterizacion.cs > {} p_mvcHacienda > PruebasCaracterizacion > ValidarYComparar(string, string, string, Func<string>): void
10 namespace p_mvcHacienda
12 public static class PruebasCaracterizacion
17 public static void EjecutarCasosASIS()
103 );
104
105 // Caso CC-08
106 ValidarYComparar(
107 id: "CC-08",
108 descripcion: "Búsqueda y consulta de atributos de Vaca1",
109 esperado: "Res encontrada: Vacal, Edad: 10, Peso: 200",
110 accion: () => {
111     Potrero p = new Potrero("P1", l_tipos_potreros.Ternero);
112     AgregarConValidacion(p, new Ternero("Vacal", 200, DateTime.Now.AddMonths(-10)));
113     Res encontrada = p.buscar_res("Vacal");
114     return $"Res encontrada: {encontrada.Nombre}, Edad: {encontrada.Edad()}, Peso: {encont
115 });
116
117 Console.WriteLine("=====\\n");
118

```

Tabla 1: Casos 1 al 8 (Auditoría de Regresión AS-IS vs TO-BE)

| Caso ID | Escena<br>rio<br>Evalua<br>do | Entradas /<br>Condición | Salida<br>Esperada<br>(Línea Base<br>AS-IS) | Salida<br>Obtenida<br>(Sistema<br>Refactorizado<br>TO-BE) | Estado<br>auditoría |
|---------|-------------------------------|-------------------------|---------------------------------------------|-----------------------------------------------------------|---------------------|
|         |                               |                         |                                             |                                                           |                     |

|              |                                        |                                                      |                                                                                        |                                                                                        |                                    |
|--------------|----------------------------------------|------------------------------------------------------|----------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------|------------------------------------|
| <b>CC-01</b> | <i>Inserción válida de Ternero</i>     | Potrero P1 (Ternero),<br>Res Vaca1 (200kg, 10 meses) | La res 'Vaca1' fue añadida al potrero 'P1'.                                            | La res 'Vaca1' fue añadida al potrero 'P1'.                                            | <b>PASÓ</b><br><b>(Preservado)</b> |
| <b>CC-02</b> | <i>Ternero con edad fuera de rango</i> | Potrero P1 (Ternero),<br>Res Vaca2 (200kg, 20 meses) | EXCEPCION:<br>La res 'Vaca2' no cumple las condiciones de peso/edad para su categoría. | EXCEPCION:<br>La res 'Vaca2' no cumple las conditions de peso/edad para su categoría.  | <b>PASÓ</b><br><b>(Preservado)</b> |
| <b>CC-03</b> | <i>Inserción válida de Cebón</i>       | Potrero P2 (Cebón),<br>Res Vaca3 (350kg, 30 meses)   | La res 'Vaca3' fue añadida al potrero 'P2'.                                            | La res 'Vaca3' fue añadida al potrero 'P2'.                                            | <b>PASÓ</b><br><b>(Preservado)</b> |
| <b>CC-04</b> | <i>Cebón con edad fuera de rango</i>   | Potrero P2 (Cebón),<br>Res Vaca4 (350kg, 5 meses)    | EXCEPCION:<br>La res 'Vaca4' no cumple las condiciones de peso/edad para su categoría. | EXCEPCION:<br>La res 'Vaca4' no cumple las condiciones de peso/edad para su categoría. | <b>PASÓ</b><br><b>(Preservado)</b> |

|              |                                          |                                                   |                                                                    |                                                                    |                          |
|--------------|------------------------------------------|---------------------------------------------------|--------------------------------------------------------------------|--------------------------------------------------------------------|--------------------------|
| <b>CC-05</b> | <i>Inserción válida de Novillo</i>       | Potrero P3 (Novillo), Res Vaca5 (450kg, 55 meses) | La res 'Vaca5' fue añadida al potrero 'P3'.                        | La res 'Vaca5' fue añadida al potrero 'P3'.                        | <b>PASÓ (Preservado)</b> |
| <b>CC-06</b> | <i>Exceso de capacidad en potrero</i>    | Potrero P4, Ingreso de 51 reses (Límite 50)       | EXCEPCION: El potrero 'P4' alcanzó su capacidad máxima (50 reses). | EXCEPCION: El potrero 'P4' alcanzó su capacidad máxima (50 reses). | <b>PASÓ (Preservado)</b> |
| <b>CC-07</b> | <i>Inserción de res con nombre vacío</i> | Potrero P1, Res sin nombre ""                     | EXCEPCION: El nombre de la res no puede estar vacío.               | EXCEPCION: El nombre de la res no puede estar vacío.               | <b>PASÓ (Preservado)</b> |
| <b>CC-08</b> | <i>Búsqueda y consulta de res</i>        | Potrero P1, Buscar Vaca1                          | Res encontrada: Vaca1, Edad: 10, Peso: 200                         | Res encontrada: Vaca1, Edad: 10, Peso: 200                         | <b>PASÓ (Preservado)</b> |

Adicionalmente a los 8 casos de regresión, se incorporaron 5 pruebas de ejecución aislada para demostrar la operabilidad y desacoplamiento de los patrones de diseño implementados:

This screenshot shows the implementation of the Strategy pattern in a C# application. The Explorer pane on the left displays the project structure, including folders for 'Bib\_Hacienda', 'Reglas', 'documentos', 'p\_mvchacienda', 'bin', 'Controllers', 'Datos', 'Imagenes', 'Infraestructura', 'Models', 'obj', 'Properties', 'Servicios', 'contratos', and 'Views'. The main editor displays the file 'PruebasCaracterizacion.cs' with the following code:

```
10 namespace p_mvchacienda
12 public static class PruebasCaracterizacion
13 {
14     1 reference
15     public static void ProbarPatronStrategyVentas()
16     {
17         Console.WriteLine("\n=====");
18         Console.WriteLine("PRUEBA DE CARACTERIZACIÓN: PATRÓN STRATEGY");
19         Console.WriteLine("=====");
20
21         try
22         {
23             Res miCebon = new Cebon("Toro Loco", 400, DateTime.Now.AddYears(-3));
24             Venta ventaToBe = new Venta(null, null, DateTime.Now, miCebon, 0);
25
26             ventaToBe.ProcesarVenta(new EstrategiaVentaCebon());
27
28             Console.WriteLine($"Monto Calculado por Strategy (TO-BE): {ventaToBe.Monto}");
29         }
30         catch (Exception ex)
31         {
32             Console.WriteLine($"ERROR EN LA PRUEBA STRATEGY: {ex.Message}");
33         }
34
35         Console.WriteLine("\n");
36     }
37 }
```

This screenshot shows the implementation of the Abstract Factory pattern in a C# application. The Explorer pane on the left displays the project structure, including folders for 'Bib\_Hacienda', 'Reglas', 'documentos', 'p\_mvchacienda', 'bin', 'Controllers', 'Datos', 'Imagenes', 'Infraestructura', 'Models', 'obj', 'Properties', 'Servicios', 'contratos', and 'Views'. The main editor displays the file 'PruebasCaracterizacion.cs' with the following code:

```
199 public static void ProbarPatronAbstractFactoryProductosGanaderos()
200 {
201     Console.WriteLine("\n=====");
202     Console.WriteLine("PRUEBA DE CARACTERIZACIÓN: PATRÓN ABSTRACT FACTORY");
203     Console.WriteLine("=====");
204
205     try
206     {
207         ProductoGanaderoFactory fabricaLeche = new LecheFactory(3.8f);
208         ProductoGanaderoFactory fabricaCarne = new CarneFactory(5, "Lomo Fino");
209         ProductoGanaderoFactory fabricaPiel = new PielFactory(3.0f, "A");
210
211         ProductoGanadero productoLeche = fabricaLeche.CrearProductoGanadero("L01", "Leche Entera",
212         ProductoGanadero productoCarne = fabricaCarne.CrearProductoGanadero("C01", "Carne Premium"
213         ProductoGanadero productoPiel = fabricaPiel.CrearProductoGanadero("P01", "Piel Curtida", 4
214
215         Console.WriteLine($"ÉXITO (Abstract Factory). Productos creados dinámicamente:\n" +
216         $" - {productoLeche.Nombre} {productoLeche.CalcularPrecio()}\n" +
217         $" - {productoCarne.Nombre} {productoCarne.CalcularPrecio()}\n" +
218         $" - {productoPiel.Nombre} {productoPiel.CalcularPrecio()}");
219     }
220     catch (Exception ex)
221     {
222         Console.WriteLine($"ERROR EN ABSTRACT FACTORY: {ex.Message}");
223     }
224
225     Console.WriteLine("\n");
226 }
```

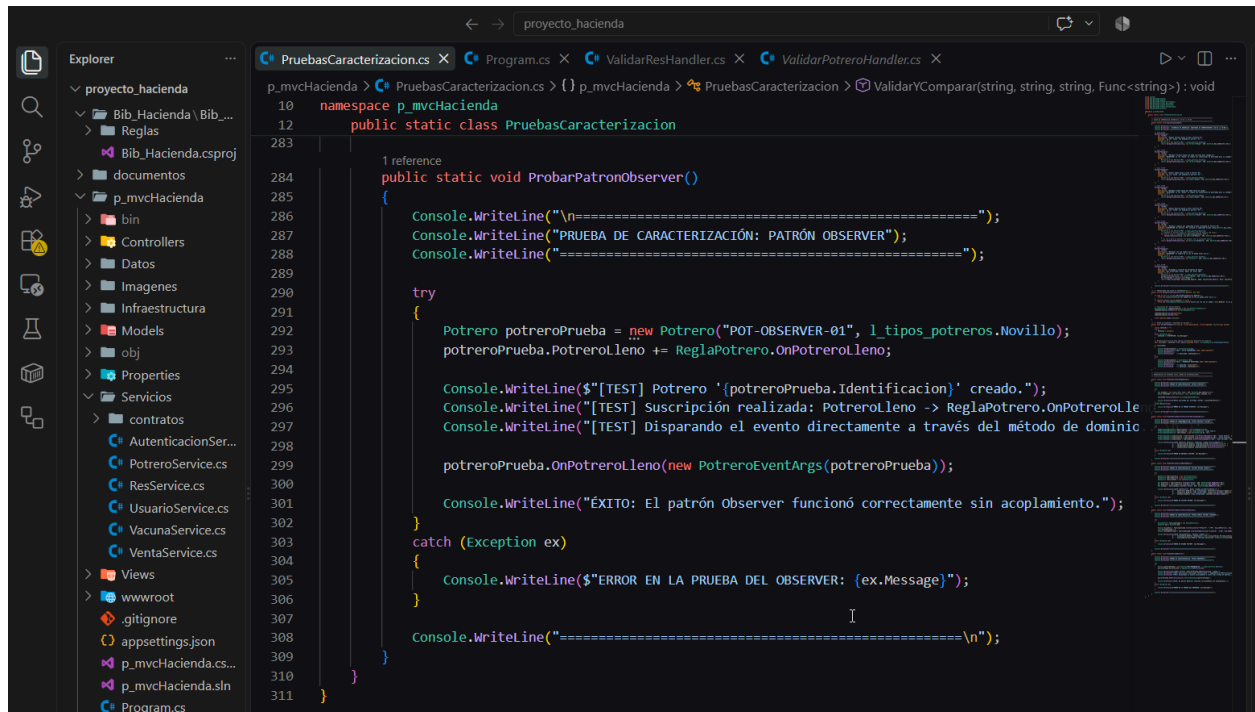


The screenshot shows the Visual Studio IDE with the file explorer on the left displaying the project structure of 'proyecto\_hacienda'. The main editor window shows the file 'PruebasCaracterizacion.cs' with the following code:

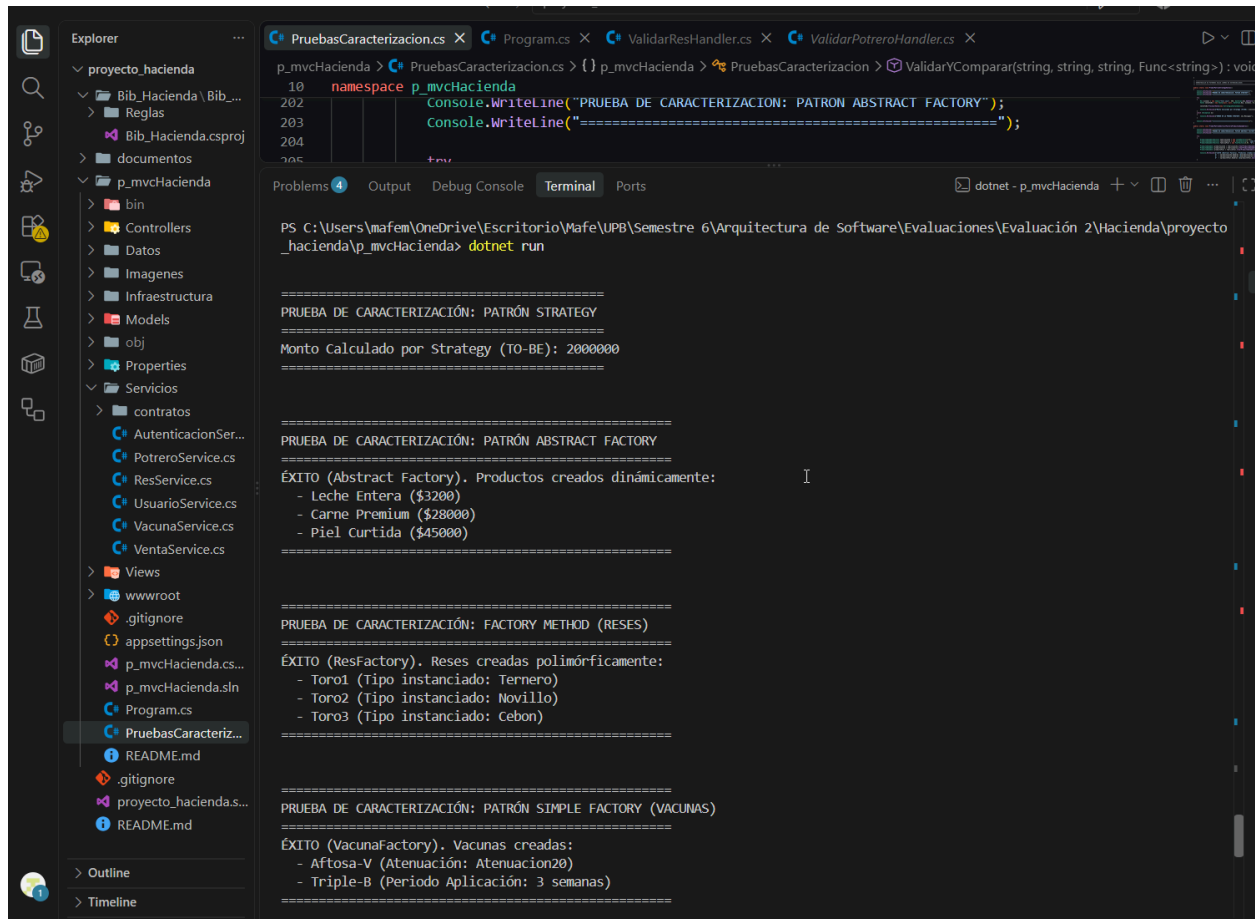
```
10 namespace p_mvchacienda
11 {
12     public static class PruebasCaracterizacion
13     {
228         public static void ProbarPatronFactoryMethodReses()
229         {
230             Console.WriteLine("\n=====");
231             Console.WriteLine("PRUEBA DE CARACTERIZACIÓN: FACTORY METHOD (RESES)");
232             Console.WriteLine("=====");
233
234             try
235             {
236                 ResFactory fabricaTernero = new TerneroFactory();
237                 ResFactory fabricaNovillo = new NovilloFactory();
238                 ResFactory fabricaCebon = new CebonFactory();
239
240                 Res miTernero = fabricaTernero.CrearRes("Toro1", 200, DateTime.Now.AddMonths(-10));
241                 Res miNovillo = fabricaNovillo.CrearRes("Toro2", 450, DateTime.Now.AddMonths(-55));
242                 Res miCebon = fabricaCebon.CrearRes("Toro3", 350, DateTime.Now.AddMonths(-30));
243
244                 Console.WriteLine($"ÉXITO (ResFactory). Reses creadas polimórficamente:\n" +
245                     $" - {miTernero.Nombre} (Tipo instanciado: {miTernero.GetType().Name})\n" +
246                     $" - {miNovillo.Nombre} (Tipo instanciado: {miNovillo.GetType().Name})\n" +
247                     $" - {miCebon.Nombre} (Tipo instanciado: {miCebon.GetType().Name})");
248             }
249             catch (Exception ex)
250             {
251                 Console.WriteLine($"ERROR EN FACTORY METHOD: {ex.Message}");
252             }
253
254             Console.WriteLine("=====\n");
255         }
256     }
257 }
```

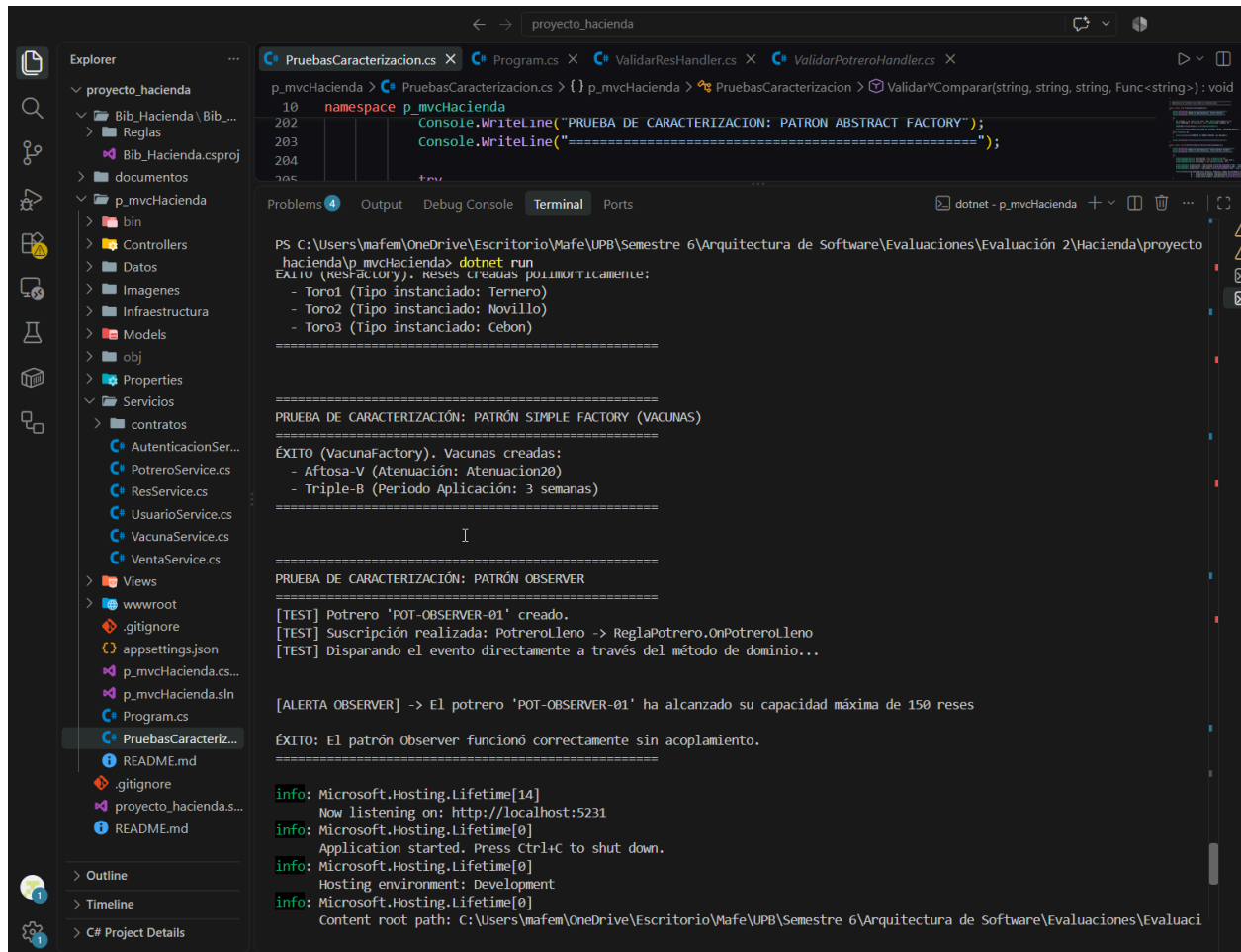
The screenshot shows the Visual Studio IDE with the file explorer on the left displaying the project structure of 'proyecto\_hacienda'. The main editor window shows the file 'PruebasCaracterizacion.cs' with the following code:

```
10 namespace p_mvchacienda
11 {
12     public static class PruebasCaracterizacion
13     {
257         public static void ProbarPatronAbstractFactoryVacunas()
258         {
259             Console.WriteLine("\n=====");
260             Console.WriteLine("PRUEBA DE CARACTERIZACIÓN: PATRÓN SIMPLE FACTORY (VACUNAS)");
261             Console.WriteLine("=====");
262
263             try
264             {
265                 VacunaFactory fabricaSanidad = new VacunaFactory();
266                 DateTime hoy = DateTime.Now;
267
268                 Vacuna vacunaViva = fabricaSanidad.CrearVacunaViva("Aftosa-V", "L-99", hoy.AddYears(1), hoy);
269                 uint periodoPrueba = 3;
270                 Vacuna vacunaBacteriana = fabricaSanidad.CrearVacunaBacteriana("Triple-B", "L-88", hoy.AddYears(1), hoy);
271
272                 Console.WriteLine($"ÉXITO (VacunaFactory). Vacunas creadas:\n" +
273                     $" - {vacunaViva.Nombre} (Atenuación: {(Vacuna)VacunaViva.Periodo_atenuacion})\n" +
274                     $" - {vacunaBacteriana.Nombre} (Periodo Aplicación: {(Vacuna)VacunaBacteriana.Periodo_aplicacion})");
275             }
276             catch (Exception ex)
277             {
278                 Console.WriteLine($"ERROR EN VACUNA FACTORY: {ex.Message}");
279             }
280
281             Console.WriteLine("=====\n");
282         }
283     }
284 }
```



Salida:





# ACTIVIDAD 5 — ANÁLISIS DE RIESGOS

## Entregable 5.1: Registro de riesgos

| ID   | Riesgo (si ocurre X, entonces Y)                                                                                | Prob (1-5) | Imp (1-5) | Exp (Pxl) | Qué hacen para evitarlo                                             | Cómo se enteran (Señal observable)              |
|------|-----------------------------------------------------------------------------------------------------------------|------------|-----------|-----------|---------------------------------------------------------------------|-------------------------------------------------|
| R-01 | Si se solicita clonar un prototipo de Res cuya llave (string) no existe en el Registro, la aplicación falla por | 2          | 4         | 8         | Mantener validaciones previas y retornar un Result o null seguro en | KeyNotFoundEx ception en los logs del servidor. |

| ID   | Riesgo (si ocurre X, entonces Y)                                                                                                                           | Prob (1-5) | Imp (1-5) | Exp (Pxl) | Qué hacen para evitarlo                                                                                     | Cómo se enteran (Señal observable)                                                      |
|------|------------------------------------------------------------------------------------------------------------------------------------------------------------|------------|-----------|-----------|-------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------|
|      | clave no encontrada.                                                                                                                                       |            |           |           | lugar de lanzar excepción.                                                                                  |                                                                                         |
| R-02 | Si un observador/suscript or falla silenciosamente, el <b>Publisher</b> podría detener la cadena de eventos o propagar la excepción al servicio principal. | 3          | 4         | 12        | Envolver la invocación a los suscriptores en bloques try/catch independientes dentro del <b>Publisher</b> . | Alertas parciales; una operación indica éxito pero el correo/log secundario no llega.   |
| R-03 | Si se clona una Res mediante Prototype sin aplicar "Deep Copy" correctamente, las listas de historial médico compartirán memoria entre vacas.              | 2          | 5         | 10        | Implementación estricta de la clonación profunda probada mediante tests unitarios.                          | Alteraciones cruzadas en el sistema (ej. vacunar una vaca afecta el historial de otra). |
| R-04 | Si se agrega un nuevo rol en la base de datos pero no se implementa su <b>IAuthStrategy</b> , el sistema denegará el acceso.                               | 3          | 3         | 9         | Configurar una estrategia "Default" que deniegue permisos y registre la anomalía temporalmente.             | Logs de "AuthStrategy no encontrada para Rol X" y rechazo en Login.                     |

## ACTIVIDAD 6 — DOS VISTAS

## 6.1 Vista para el Negocio

- **¿Qué le vamos a hacer al sistema y qué NO cambia?**

El objetivo de este cambio es preparar el sistema para que pueda incorporar nuevas necesidades de la hacienda con menor impacto sobre las funcionalidades que ya están funcionando. Actualmente, algunas decisiones se encuentran concentradas en pocos puntos del sistema, por lo que la incorporación de nuevos tipos de animales, nuevas validaciones, nuevos roles o nuevas reacciones ante eventos puede obligar a modificar varias partes existentes.

La propuesta reorganiza internamente estas decisiones para que cada nueva necesidad pueda incorporarse en un punto específico, reduciendo la cantidad de funcionalidades existentes que deben ser modificadas. También se separan las diferentes reglas de validación, las formas de autorización y las reacciones ante determinados acontecimientos, de manera que puedan evolucionar de forma independiente.

No cambian las reglas actuales del negocio, los cálculos, las operaciones existentes ni los resultados que recibe el usuario. El alcance se limita a la forma en que el sistema organiza internamente estas responsabilidades. El comportamiento observable que ya fue validado en el Reto 1 permanece congelado. El reto exige precisamente demostrar mediante los casos anteriores y nuevos que las salidas continúan coincidiendo.

- **¿Dónde se está yendo el tiempo/dinero hoy?**

El principal costo actual no está en que una operación individual sea lenta, sino en la cantidad de elementos que deben revisarse y modificarse cada vez que aparece una nueva necesidad.

El análisis realizado sobre la situación actual permitió medir este costo:

| Situación actual                        | Elementos que pueden requerir modificación | Impacto                                                                        |
|-----------------------------------------|--------------------------------------------|--------------------------------------------------------------------------------|
| Incorporar un nuevo tipo de animal      | 5                                          | Una decisión de creación está distribuida entre diferentes partes del sistema. |
| Incorporar una nueva variante de vacuna | 6                                          | La información debe reconocerse explícitamente en diferentes puntos.           |
| Agregar una nueva                       | 7                                          | La estructura actual obliga                                                    |

|                                              |   |                                                             |
|----------------------------------------------|---|-------------------------------------------------------------|
| validación                                   |   | a intervenir varias validaciones existentes.                |
| Incorporar una nueva reacción ante un evento | 5 | La coordinación está concentrada en los servicios actuales. |

Estas mediciones provienen del análisis de los puntos de dolor realizado sobre el sistema actual. Por ejemplo, agregar un nuevo tipo de animal requiere actualmente modificar cuatro elementos existentes y crear uno nuevo, mientras que una nueva validación puede involucrar seis elementos existentes y uno adicional.

Esto significa que una solicitud de cambio de negocio puede terminar convirtiéndose en una intervención sobre múltiples partes que ya funcionan. Cada elemento adicional que debe revisarse aumenta el esfuerzo de desarrollo, las pruebas necesarias y las posibilidades de afectar accidentalmente una operación existente.

Por esta razón, el problema no es únicamente técnico: el costo de evolución del producto aumenta cada vez que el negocio necesita una nueva capacidad.

- **¿Qué gana el negocio (Tiempos de respuesta y riesgos)?**

El principal beneficio es reducir el impacto de los cambios futuros.

Con la nueva organización, las solicitudes de cambio podrán concentrarse en los puntos responsables de cada decisión, en lugar de obligar a modificar repetidamente funcionalidades existentes. Esto permite obtener tres beneficios principales:

**1. Menor tiempo para atender nuevas solicitudes:** Cuando se solicite una nueva capacidad, el equipo tendrá un lugar definido donde incorporar esa variación. Esto disminuye la cantidad de componentes existentes que deben revisarse antes de realizar el cambio. El objetivo no es prometer una cantidad determinada de horas de ahorro —esa medición todavía no hace parte de las evidencias disponibles—, sino reducir objetivamente la cantidad de elementos que deben modificarse para atender una solicitud.

**2. Menor riesgo de afectar funcionalidades existentes:** Una modificación concentrada en un punto específico reduce la superficie que debe tocarse. Esto facilita comprobar que las operaciones que ya funcionan continúan entregando los mismos resultados.

**3. Mayor capacidad para responder a nuevas necesidades del negocio:** La hacienda puede requerir nuevos tipos de animales, nuevas formas de validar información, nuevos roles o nuevas acciones cuando ocurren determinados acontecimientos. La propuesta busca que estas necesidades puedan incorporarse sin convertir cada nueva solicitud en

una modificación extensa del sistema.

- **¿Qué cuesta y qué riesgos hay en operación?**

La mejora no es gratuita. La nueva organización aumenta inicialmente la cantidad de componentes internos y los pasos que debe seguir un desarrollador para comprender cómo se ejecuta una operación.

Esto tiene tres costos principales:

- Hay más elementos que mantener.
- Algunas operaciones tienen un nivel adicional de intermediación.
- Para diagnosticar ciertos problemas puede ser necesario seguir una secuencia de componentes en lugar de encontrar toda la lógica en un único lugar.

También existen riesgos asociados al cambio. Uno de ellos es que una nueva forma de creación sea solicitada para un elemento que no esté registrado correctamente, provocando un fallo al intentar localizarlo. Otro es que una notificación secundaria falle mientras la operación principal continúa, haciendo que una alerta o registro esperado no se produzca. También existe el riesgo de compartir información entre animales cuando se realizan copias incorrectas, lo que podría provocar que un cambio realizado sobre un animal afecte la información de otro. Finalmente, un nuevo rol podría quedar sin una regla de autorización asociada y ser rechazado por el sistema. Estos riesgos ya están identificados en el registro de riesgos del proyecto. Para controlar estos riesgos se plantean validaciones, pruebas específicas, manejo seguro de errores y señales observables en los registros del sistema.

- **¿Qué necesitamos de ustedes y qué pasa si no se hace?**

Para realizar el cambio necesitamos que el negocio mantenga definidos y aprobados los comportamientos que deben conservarse. No se requiere modificar las reglas actuales ni aprobar nuevas funcionalidades como parte de esta intervención. También necesitamos que las futuras solicitudes de cambio sean expresadas como necesidades concretas del negocio, de manera que puedan incorporarse en los puntos correspondientes sin alterar funcionalidades que no hacen parte de la solicitud. Si este trabajo no se realiza, el sistema continuará funcionando, pero cada nueva necesidad tendrá un costo de evolución potencialmente mayor, porque será necesario intervenir nuevamente sobre partes que ya existen.

El riesgo a mediano plazo es que una solicitud aparentemente pequeña —por ejemplo, incorporar un nuevo tipo de animal, una nueva validación o un nuevo rol— termine requiriendo modificaciones en varios componentes existentes. Esto aumenta progresivamente el esfuerzo de desarrollo y pruebas y hace más probable que un cambio futuro afecte una funcionalidad que anteriormente funcionaba correctamente. Por tanto, la decisión no busca cambiar lo que el sistema hace hoy, sino disminuir el costo y el riesgo



de cambiarlo mañana.

## 6.2 Vista para el Equipo de Desarrollo (Guía Rápida)

Esta guía permite a un desarrollador incorporarse al proyecto y localizar rápidamente dónde realizar cambios relacionados con la creación de animales, validaciones, cálculo de ventas y eventos, sin modificar componentes que no corresponden a la solicitud.

El diseño TO-BE incorpora cuatro patrones: Factory Method, Chain of Responsibility, Strategy y Observer. Cada uno responde a un punto de dolor identificado en el diseño AS-IS y tiene un límite específico que debe respetarse. La arquitectura mantiene la separación existente entre presentación, aplicación, dominio e infraestructura.

### Matriz: Dónde tocar para futuros cambios

| Patrón                  | Punto de dolor                                 | Donde vive                     | Responsabilidad                                       | Se relaciona con                                       |
|-------------------------|------------------------------------------------|--------------------------------|-------------------------------------------------------|--------------------------------------------------------|
| Factory Method          | P-01 — creación rígida de subtipos de Res      | Zona de creación de Res        | Encapsular la creación de cada subtipo                | ResController y subtipos de Res                        |
| Chain of Responsibility | P-03 — interfaz monolítica de validación       | Validaciones/                  | Ejecutar validaciones independientes en cadena        | Servicios que requieren validar información            |
| Strategy                | P-05 — cálculo condicionado del monto de venta | Venta / estrategias de cálculo | Encapsular la fórmula de cálculo según el tipo de Res | Venta, IEstrategiaCalculoVenta y estrategias concretas |
| Observer                | P-04 — coordinación manual de eventos          | Servicios y publishers         | Notificar eventos a múltiples consumidores            | ResService, PotreroService, VacunaService y observers  |

### Guía de dónde tocar

| Tipo de cambio    | Que debo crear                            | Que debo modificar                    | Que NO debo tocar                           |
|-------------------|-------------------------------------------|---------------------------------------|---------------------------------------------|
| Nuevo tipo de Res | Nueva implementación concreta de Res y su | Registro/configuración de la creación | ResController para agregar nuevos switch/if |

|                                              |                                                                                 |                                                                               |                                                           |
|----------------------------------------------|---------------------------------------------------------------------------------|-------------------------------------------------------------------------------|-----------------------------------------------------------|
|                                              | mecanismo de creación                                                           |                                                                               |                                                           |
| Nueva validación                             | Nuevo validador                                                                 | Configuración de la cadena en Program.cs                                      | Validadores existentes que no cambian                     |
| Nueva regla de cálculo de venta              | Nueva implementación de IEstrategiaCalculoVenta con la fórmula correspondiente. | Configuración/asociación de la estrategia con el tipo de Res correspondiente. | La lógica central de Venta para agregar nuevos if/switch. |
| Nueva reacción ante un evento                | Nuevo Observer                                                                  | Suscripción/configuración del observer                                        | Lógica principal del servicio que publica el evento       |
| SC-1 — Nuevos productos derivados del ganado | Nueva implementación de producto según el tipo requerido                        | Registro/creación y persistencia correspondiente                              | Lógica existente de Res que no participe en la venta      |
| SC-2 — Chips de geolocalización              | Componentes correspondientes a geolocalización                                  | Servicio y persistencia de geolocalización                                    | Servicios no relacionados con ubicación                   |
| SC-3 — Historia clínica                      | HistoriaClinica / RegistroClinico cuando corresponda al cambio                  | Servicio y persistencia de historia clínica                                   | VacunaService y otras responsabilidades no relacionadas   |